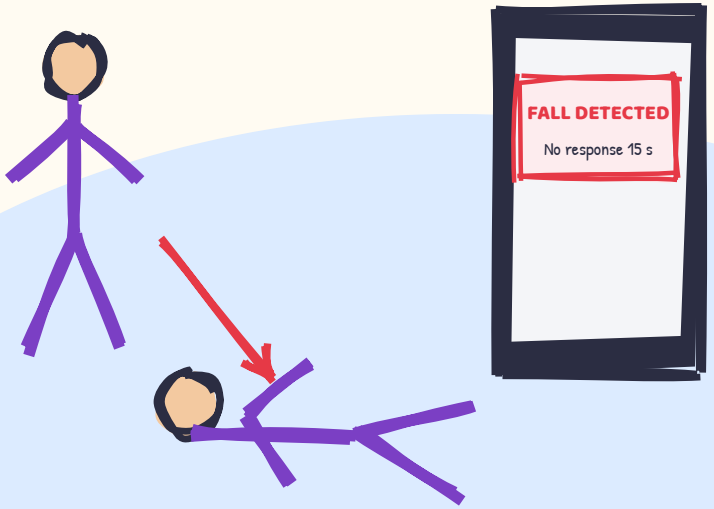
Tech Leaders · Class 4

Fall Detection Wearable for the Elderly

Spot a fall, wait for 'I'm OK', then call for help

MPU6050 motion | State machine | Wi-Fi phone alert



Did you know?
Some smartwatches already detect hard falls and call for help if the wearer does not respond.

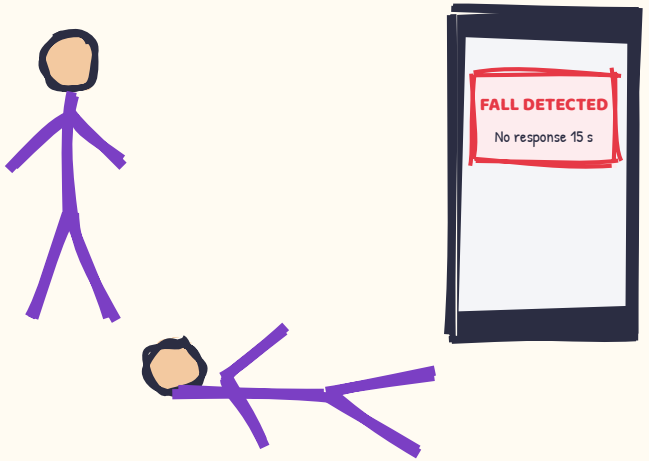
LEARN

Engineering for people

What if nobody is home?

A real worry

Many grandparents live alone or are alone for hours while the family is at work. After a fall they may not be able to reach a phone. Getting help **quickly** makes a big difference.



Phone them every hour?

A camera in every room?

A small wearable that calls for help!

Today: you build a wearable that recognises a fall, gives the wearer **15 seconds to cancel**, then sends an alert to a caregiver's phone.

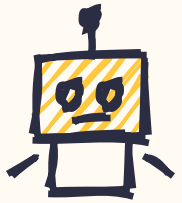
LEARN

Learning aims

Today I will learn to...

1. Read 3-axis acceleration and calculate its **magnitude in g**
2. **Collect and label data** for walking, sitting and simulated falls
3. Detect a fall with a **state machine**: impact → stillness → alarm
4. Send a **Wi-Fi push alert** after a cancel window
5. Measure **accuracy and false alarms** and design ethically

Keywords
acceleration · g · magnitude · threshold · state machine · false positive · sensitivity · consent



LEARN

Watch and think

Video time!



Hard Fall Detection Explained

Wearholic · about 3 minutes · youtu.be/mLNfA7WyMNM

Watch for...

1. Which **sensor** notices a fall?
2. What does the watch do **after** a fall?
3. How does it avoid false alarms?

Scan to watch

After the video: Design a rule: what pattern of acceleration means 'fall', and what means 'sat down quickly'?

Real world

Fall detection today



Smartwatches
Detect hard falls and offer to call emergency services



Alert pendants
Worn by older people living alone, with a help button



Hospitals & care homes
Bed and floor sensors alert nurses



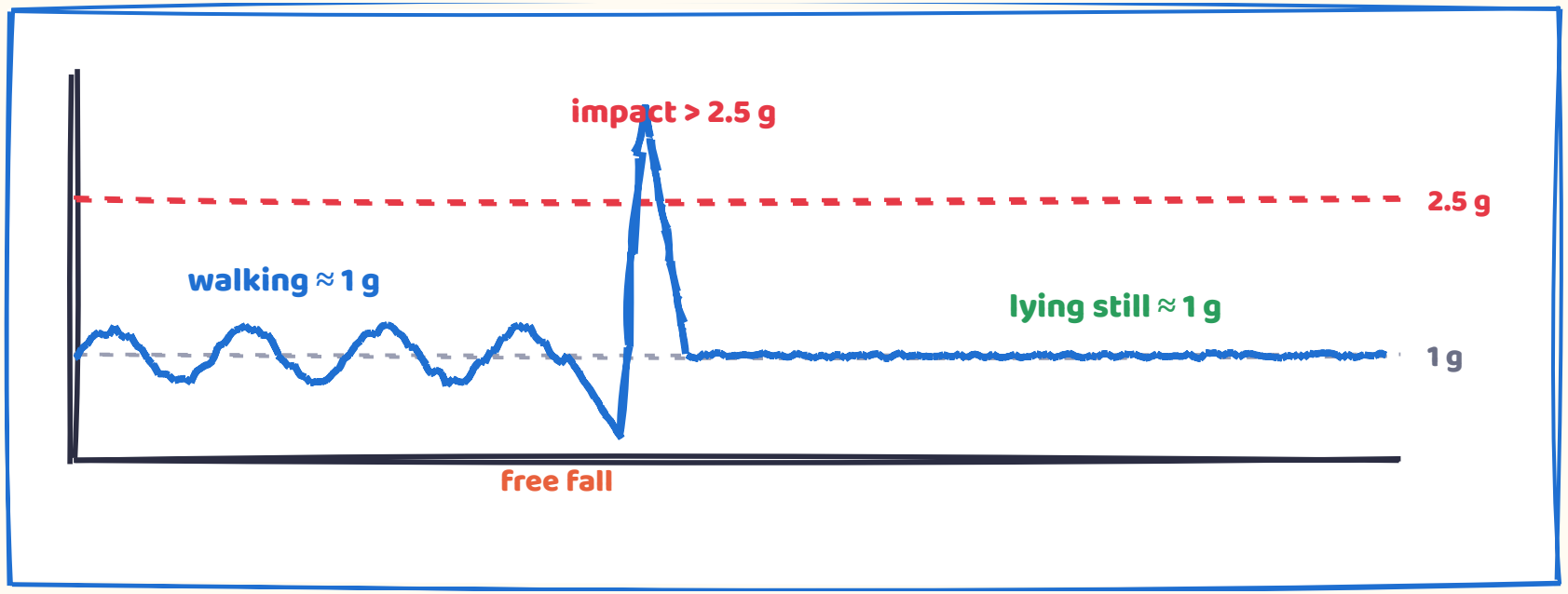
Workers
Lone workers at heights wear 'man-down' alarms

? Why might an older person refuse to wear a device like this? How could design fix that?

Design truth
The best device is the one people actually wear. Comfort and dignity matter as much as code.

The physics

What a fall looks like to an accelerometer



1 g
= 9.81 m/s². A hard landing can briefly reach several g.

Real falls vary
Slow slides down a wall have small spikes. No algorithm catches every fall.

Magnitude

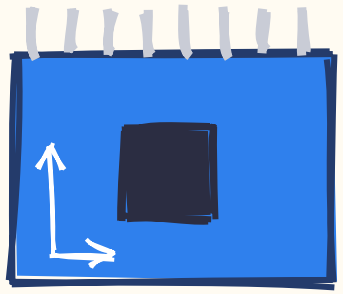
|a| = √(ax² + ay² + az²). Standing still it reads **1 g** (gravity) in any orientation, so we don't care how the device is turned.

Fall signature

A short drop (below 1 g), a hard **impact spike**, then **stillness** in a lying position.

Meet the sensor

MPU6050: accelerometer + gyroscope



X Y Z axes

Set the range to ±8 g!

The default range is ±2 g. A fall impact is bigger, so the reading **clips** at 2 g and every bump looks the same. ±8 g sees the real spike.

Library

Adafruit MPU6050 (installs Adafruit Unified Sensor too). Gives acceleration in m/s²: divide by 9.81 for g.

MPU6050	ESP32
VCC	3V3
GND	GND
SDA	GPIO 21
SCL	GPIO 22

I2C

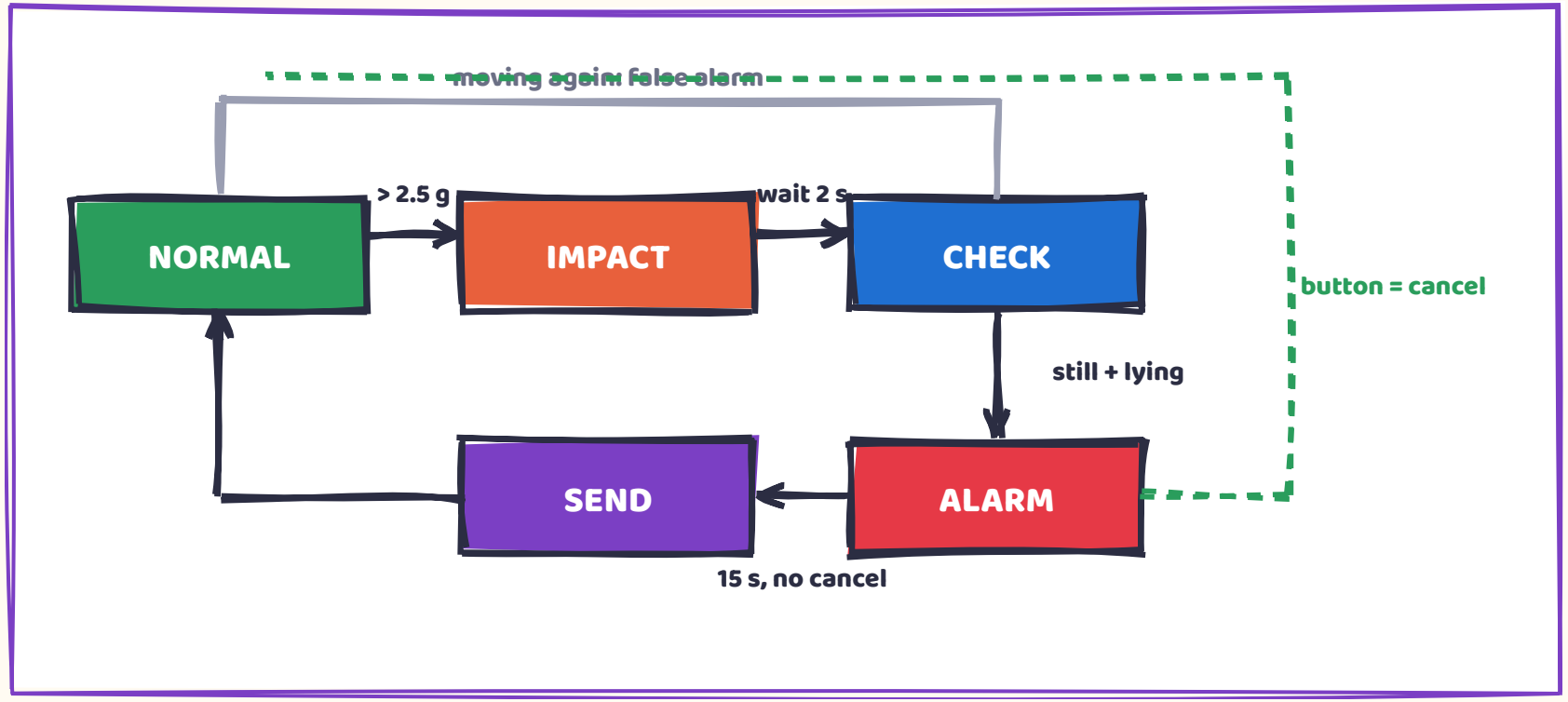
Two wires (SDA data, SCL clock) shared by many sensors. Address 0x68.

Wokwi

Practise in the Wokwi simulator first: it has an MPU6050 you can tilt with sliders.

Big idea

A state machine, not one big if-statement



Why states?

A fall is a **story over time**: impact, then stillness. The program must remember which part of the story it is in.

Free-fall too?

Adding a free-fall stage before impact cuts false alarms but misses slow falls. Trade-off!

Evaluate

How good is your detector?

Test	Trials	Alarms	Expected
Simulated fall (toy)	10	____	should alarm
Sit down fast	10	____	should NOT
Jump on the spot	10	____	should NOT
Walk 20 steps	10	____	should NOT
Lie down slowly	10	____	should NOT

Sensitivity

falls detected ÷ falls tried. 9 of 10 = **90%**.

False alarms

alarms on non-falls ÷ non-fall trials. 2 of 40 = **5%**.

The trade-off

Lower thresholds catch more falls but cause more false alarms. Which is worse for a grandparent?

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Nobody falls on purpose in this class
2. The 15-second 'I'm OK' window
3. Kit per group
4. Wire the motion sensor
5. Add the buzzer, button and light
6. Read acceleration in g
7. Collect and label data

Test safely and ethically

Nobody falls on purpose in this class

Never real falls. Simulate: strap the wearable to a soft toy and drop it onto a mattress from about 50 cm.

Sitting and jumping tests are fine on a mat, done carefully, to check for false alarms.

Consent: a real wearer must agree, understand what is sent and who receives it.

Dignity: make it discreet and comfortable. Nobody wants to wear a 'patient' badge.

Not a medical device. Our prototype supports, never replaces, care and real alarm systems.



Battery

A dead wearable protects nobody. Plan charging reminders and a low-battery alert.

Human-centred design

The 15-second 'I'm OK' window

Why a cancel window?

People sit down hard, drop the device or trip without getting hurt. Sending an alert every time causes **alarm fatigue**: caregivers start ignoring alerts.

Make it easy to cancel

A **big** button, a loud beeping countdown and a flashing light. Older users may have weak eyesight, hearing or grip.



ntfy.sh alerts

A free push service. The caregiver installs the **ntfy** app and subscribes to a secret topic name. The ESP32 posts a message to that topic.

Privacy

Anyone who guesses the topic can read it. Use a long random name and never send names or health details.

Build it

Kit per group



ESP32 DevKit
+ USB cable



MPU6050 (GY-521)
accelerometer + gyro



Buzzer
countdown beeps



Big push button
the 'I'm OK' button



USB power bank
or 18650 + TP4056 (teacher)



Phone with ntfy
the caregiver

Also: breadboard, jumpers, elastic strap or belt clip, a soft toy and a mattress or cushion for drop tests, laptop with Arduino IDE 2 + ESP32 boards + Adafruit MPU6050 library.

Power choice
A small USB power bank is simplest and safest for a wearable prototype.

MAKE

Build step 1 of 6

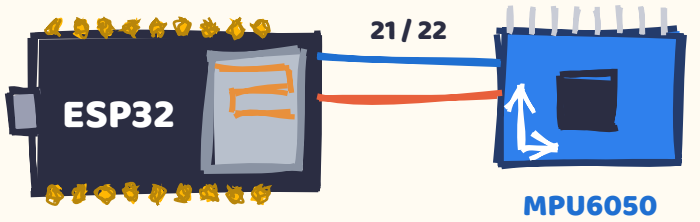
Wire the motion sensor

a MPU6050 **VCC → 3V3**, **GND → GND**, **SDA → GPIO 21**, **SCL → GPIO 22**.

b Place the sensor flat with the **Z arrow pointing up**. Note how it will sit on the body.

c Upload an I2C scanner sketch: it should report a device at **0x68**.

d Keep the sensor wires short and firm: loose wires create fake 'impacts'.



Check: the scanner finds 0x68.

0x69? The AD0 pin is high: fine, but change the address in the code.

MAKE

Build step 2 of 6

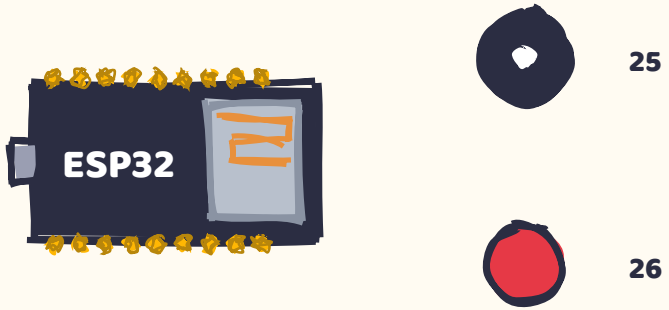
Add the buzzer, button and light

a Buzzer **+** → **GPIO 25**, **-** → GND.

b Button between **GPIO 26** and **GND**. We use the ESP32's internal pull-up (INPUT_PULLUP), so pressed = LOW.

c The built-in LED on **GPIO 2** will flash during the countdown.

d Quick test: a sketch that beeps while the button is NOT pressed.



Check: beeping stops while you hold the button.

Build step 3 of 6

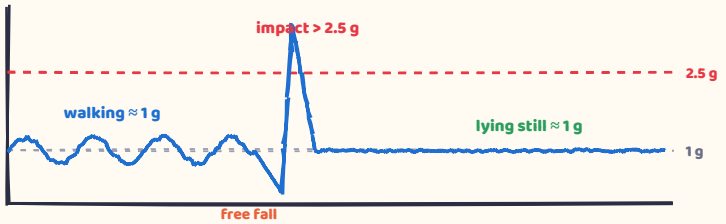
Read acceleration in g

a Install **Adafruit MPU6050** from the Library Manager (accept its dependencies).

b Upload part 1 of the code. Open the **Serial Plotter** at 115200.

c Lying still: magnitude ≈ **1.0 g** in every orientation. Turn it on its side: still ≈ 1.0.

d Tap the sensor on the table: you should see short spikes above 2 g.



Check: the still reading is between 0.95 and 1.05 g.

Reads about 9.8 instead of 1? You forgot to divide by 9.81.

Build step 4 of 6

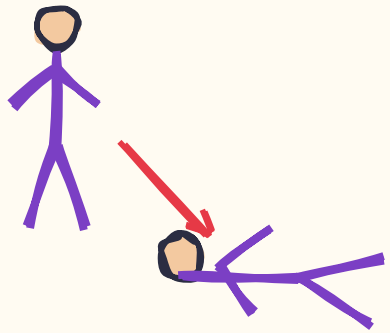
Collect and label data

a Strap the wearable to the waist (belt) or chest. Record 20 s of each: **walking**, **sitting down fast**, **jumping**, **lying down**.

b Simulated fall: strap it to a soft toy and drop it onto a mattress from 50 cm, 5 times.

c For each, write down the **peak g** and what happens after the peak.

d Choose thresholds that separate falls from everything else. That is data-driven design!



Check: you have a table of peak values for at least 4 activities.

Sitting down hard can reach 2-3 g. That is why we also check for stillness.

Build step 5 of 6

Set up the caregiver alert

a Install the free **ntfy** app on the caregiver's phone (Android or iPhone).

b Invent a long secret topic, e.g. **ptm-fall-7k3q9x**. Subscribe to it in the app.

c Test from a laptop browser or terminal: send a message to **ntfy.sh/your-topic**. It should pop up on the phone.

d Put the same topic in the code. Never share it publicly.



Check: a test message arrives on the phone within a few seconds.

School Wi-Fi blocking it? Test on a phone hotspot.

MAKE

Code it: part 1

Set up and read g

```
#include <Adafruit_MPU6050.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>

Adafruit_MPU6050 mpu;
const int BUZZ = 25, BTN = 26, LED = 2;

void setup() {
  Serial.begin(115200);
  pinMode(BUZZ, OUTPUT); pinMode(LED, OUTPUT);
  pinMode(BTN, INPUT_PULLUP);            // pressed = LOW
  if (!mpu.begin()) { Serial.println("No MPU6050!"); while (1); }
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);   // see real impacts
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
  WiFi.begin("SchoolWiFi", "********");
}

float readG(float &tilt) {               // returns |a| in g
  sensors_event_t a, g, t;
  mpu.getEvent(&a, &g, &t);
  float x = a.acceleration.x / 9.81, y = a.acceleration.y / 9.81,
        z = a.acceleration.z / 9.81;
  float mag = sqrt(x * x + y * y + z * z);
  tilt = acos(constrain(y / mag, -1.0, 1.0)) * 180 / PI;  // angle from upright
  return mag;
}
```

Magnitude

One number that ignores orientation: perfect for 'how hard was that?'

Tilt

The angle between the body's 'up' axis and gravity. Standing ≈ 0°, lying ≈ 90°. Use the axis that points up when YOU wear it (here: y).

while(1)

If the sensor is missing, stop and say so. Silent failure in a safety device is dangerous.

MAKE

Code it: part 2

The fall state machine

```
enum State { NORMAL, IMPACT, ALARM };
State state = NORMAL;
unsigned long tEvent = 0;
float moveSum = 0; int samples = 0;

void loop() {
  float tilt, g = readG(tilt);
  switch (state) {
    case NORMAL:
      if (g > 2.5) { state = IMPACT; tEvent = millis(); moveSum = 0; samples = 0; }
      break;
    case IMPACT:                                  // watch for 2 s
      if (millis() - tEvent > 1000) { moveSum += fabs(g - 1.0); samples++; }
      if (millis() - tEvent > 2000) {
        bool still = samples > 0 && moveSum / samples < 0.15;
        bool lying = tilt > 60;
        if (still && lying) { state = ALARM; tEvent = millis(); }
        else state = NORMAL;                      // got up: no alarm
      }
      break;
    case ALARM:
      countdown();
      break;
  }
  delay(20);                                      // about 50 readings/s
}
```

Thresholds from YOUR data

2.5 g, 0.15 and 60° are starting values. Replace them with what your data table showed.

Stillness test

Average movement away from 1 g during seconds 1-2 after impact. Small = not moving.

Non-blocking

No long delay() calls: the loop keeps running, so the button always responds.

Code it: part 3

Countdown, cancel and alert

```
void countdown() {
  bool beepOn = (millis() / 250) % 2;              // beep 2x per second
  digitalWrite(BUZZ, beepOn); digitalWrite(LED, beepOn);
  if (digitalRead(BTN) == LOW) {                   // "I'm OK"
    digitalWrite(BUZZ, LOW); digitalWrite(LED, LOW);
    state = NORMAL; return;
  }
  if (millis() - tEvent > 15000) {                 // no answer in 15 s
    digitalWrite(BUZZ, LOW);
    sendAlert();
    state = NORMAL;
  }
}

void sendAlert() {
  if (WiFi.status() != WL_CONNECTED) return;       // (challenge: retry!)
  WiFiClientSecure client;
  client.setInsecure();                            // classroom shortcut
  HTTPClient http;
  http.begin(client, "https://ntfy.sh/ptm-fall-7k3q9x");
  http.addHeader("Title", "FALL DETECTED");
  http.addHeader("Priority", "urgent");
  int code = http.POST("Possible fall. No response for 15 seconds.");
  Serial.println(code);                            // 200 = sent
  http.end();
}
```

Cancel first

The button is checked on every loop, so a press always wins over the timer.

setInsecure()

Skips checking the server's certificate. Fine for a class demo; a real product must verify it.

What if Wi-Fi is down?

Right now the alert is lost. A real device retries, keeps beeping and stores the event.

MAKE

Build step 6 of 6

Make it wearable and test

a Move to battery power (power bank). Fix everything in a small box with the **button on top**.

b Strap it firmly at the **waist**, sensor upright. A loose device bounces and creates false spikes.

c Run the test plan on the next slide: falls with the soft toy, plus sitting, jumping and walking with a student.

d Record every result honestly, including misses.



Check: the caregiver phone receives an alert for a simulated fall that is not cancelled.

Check the 'up' axis again after boxing it: the mounting may have rotated it.

MAKE

Improve it

Fewer false alarms, fewer misses

Ideas to test (one at a time!)

- [x] Require a free-fall dip (below 0.5 g) just before the impact
- [x] Use the gyroscope: falls involve fast rotation
- [x] Check that the person stays lying for 5 s, not 2 s
- [x] Wear position: waist is usually more reliable than the wrist
- [x] Ask a few users to wear it for an hour: count real false alarms
- [x] Add a 'help' button for falls it misses

Machine learning

Instead of hand-made thresholds, train a model on your labelled data (TinyML in Phase 2, TL-19).

Report

Write: our sensitivity, our false-alarm rate, what we changed and why.

Test like an engineer

Troubleshooting

Symptom	Likely cause / fix
Magnitude stuck at 2 g on impacts	range still ±2 g: set MPU6050_RANGE_8_G
Alarm while walking	threshold too low or loose strap
Never alarms	tilt uses the wrong axis for your mounting; print tilt
No phone alert	Wi-Fi blocked, wrong topic, or not subscribed in the app
'No MPU6050!'	SDA/SCL swapped or loose; run the I2C scanner

Debug checklist

Print state, g and tilt every loop while testing.
Test each stage alone: reading, impact, stillness, countdown, alert.

Keep a test log with date, thresholds and results. Engineers prove safety with evidence.

Level up!

Level up

Choose your challenge



Never lose an alert: retry until sent; if Wi-Fi is down, keep beeping and save the event.



Battery life: measure current and estimate hours per charge; add a low-battery alert.



Daily check-in: send 'all OK' once a day so the family knows the device is working.



AI version: train a TinyML model on your labelled data and compare it with your thresholds.

Think like an engineer

Who should be able to see the fall alerts and the activity data? Write a short privacy policy for your device.

Boss level

Use the gyroscope plus accelerometer together (sensor fusion) for a more reliable detector.

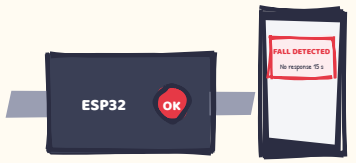
Show what you know

What did we learn?

- ☑ $|a| = \sqrt{(x^2 + y^2 + z^2)}$; still ≈ 1 g in any orientation
- ☑ A fall = impact spike, then stillness while lying
- ☑ A state machine tracks events over time
- ☑ A cancel window prevents alarm fatigue
- ☑ Measure sensitivity and false alarms, and design with consent

At home

Talk to an older relative: what would make them happy to wear a device like this?



Exit ticket: why did we change the MPU6050 range from ±2 g to ±8 g?

Next class: ESP-NOW Wireless Sensor Network!